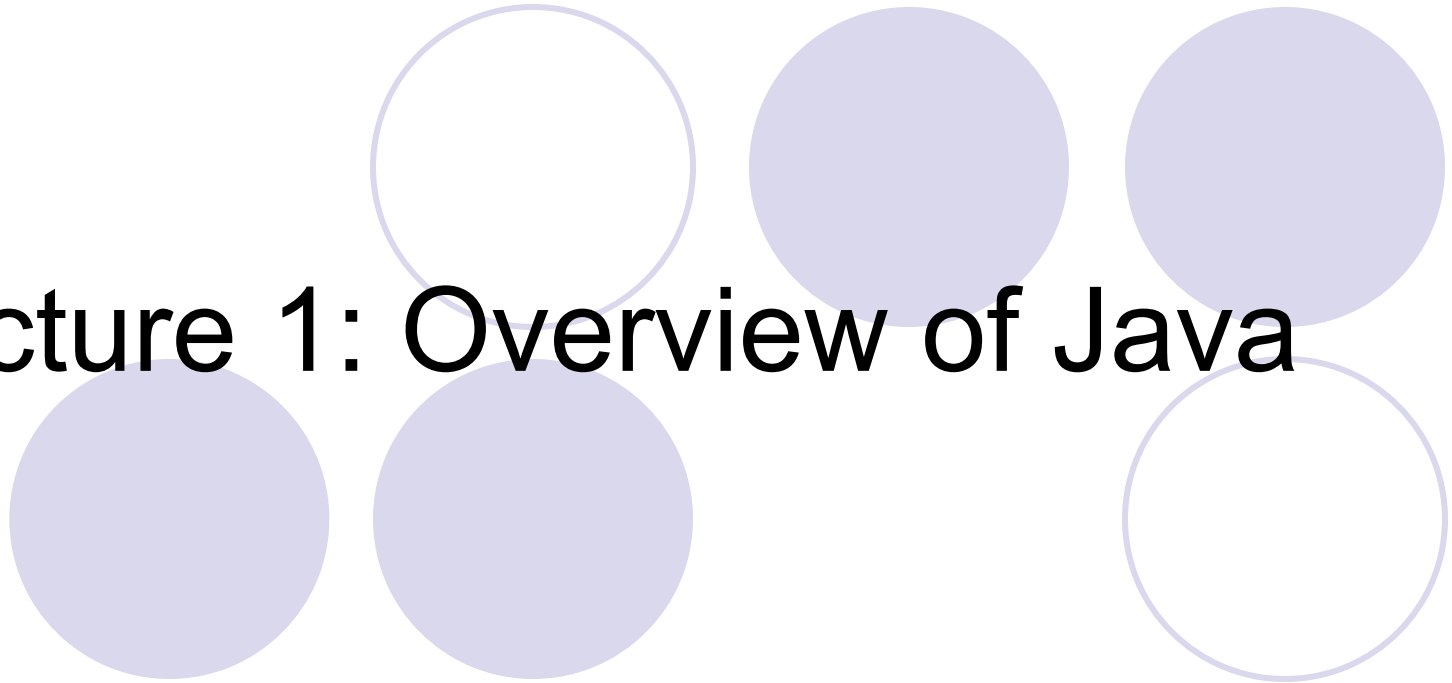
# Object Oriented Programming with JAVA

**Tasniah Mohiuddin**
**Assistant Professor, Dept. of CSE**
**Bangladesh University of Business and Technology**

# Lecture 1: Overview of Java



# What is java?

- Java is a high-level programming language originally developed by Sun Microsystems.
- Initiated by James Gosling and released in 1995 as core component of Sun Microsystems' Java platform (Java 1.0 [J2SE]).
- The latest release of the Java Standard Edition is Java SE 15.

# What is java?

- Multiple configurations were built to suit various types of platforms. For example: J2EE for Enterprise Applications, J2ME for Mobile Applications.
- A general-purpose object-oriented language
- Based on C/C++
- Designed for easy Web/Internet applications
- Widespread acceptance

# Java Features (1)

- **Object Oriented** − In Java, everything is an Object. Java can be easily extended since it is based on the Object model. 4 main concepts of Object Oriented programming are:
- **Abstraction**
- **Encapsulation**
- **Inheritance**
- **Polymorphism**
- **Platform Independent** − Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform independent byte code. This byte code is distributed over the web and interpreted by the Virtual Machine (JVM) on whichever platform it is being run on.

- **Simple** − Java is designed to be easy to learn. If you understand the basic concept of OOP Java, it would be easy to master.

# Java Features (2)

- **Secure** − With Java's secure feature it enables to develop virus-free, tamper-free systems. Authentication techniques are based on public-key encryption.

- **Architecture-neutral** − Java compiler generates an architecture-neutral object file format, which makes the compiled code executable on many processors, with the presence of Java runtime system.

- **Portable** − Being architecture-neutral and having no implementation dependent aspects of the specification makes Java portable. Compiler in Java is written in ANSI C with a clean portability boundary, which is a POSIX subset.

# Java Features (3)

- **Robust** − Java makes an effort to eliminate error prone situations by emphasizing mainly on compile time error checking and runtime checking.

- **Multithreaded** − With Java's multithreaded feature it is possible to write programs that can perform many tasks simultaneously. This design feature allows the developers to construct interactive applications that can run smoothly.

- **Interpreted** − Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and light-weight process.

# Java Features (4)

- **High Performance** − With the use of Just-In-Time compilers, Java enables high performance.

- **Distributed** − Java is designed for the distributed environment of the internet.

- **Dynamic** − Java is considered to be more dynamic than C or C++ since it is designed to adapt to an evolving environment.
- Java programs can carry extensive amount of run-time information that can be used to verify and resolve accesses to objects on run-time.

# Java Virtual Machine (JVM)

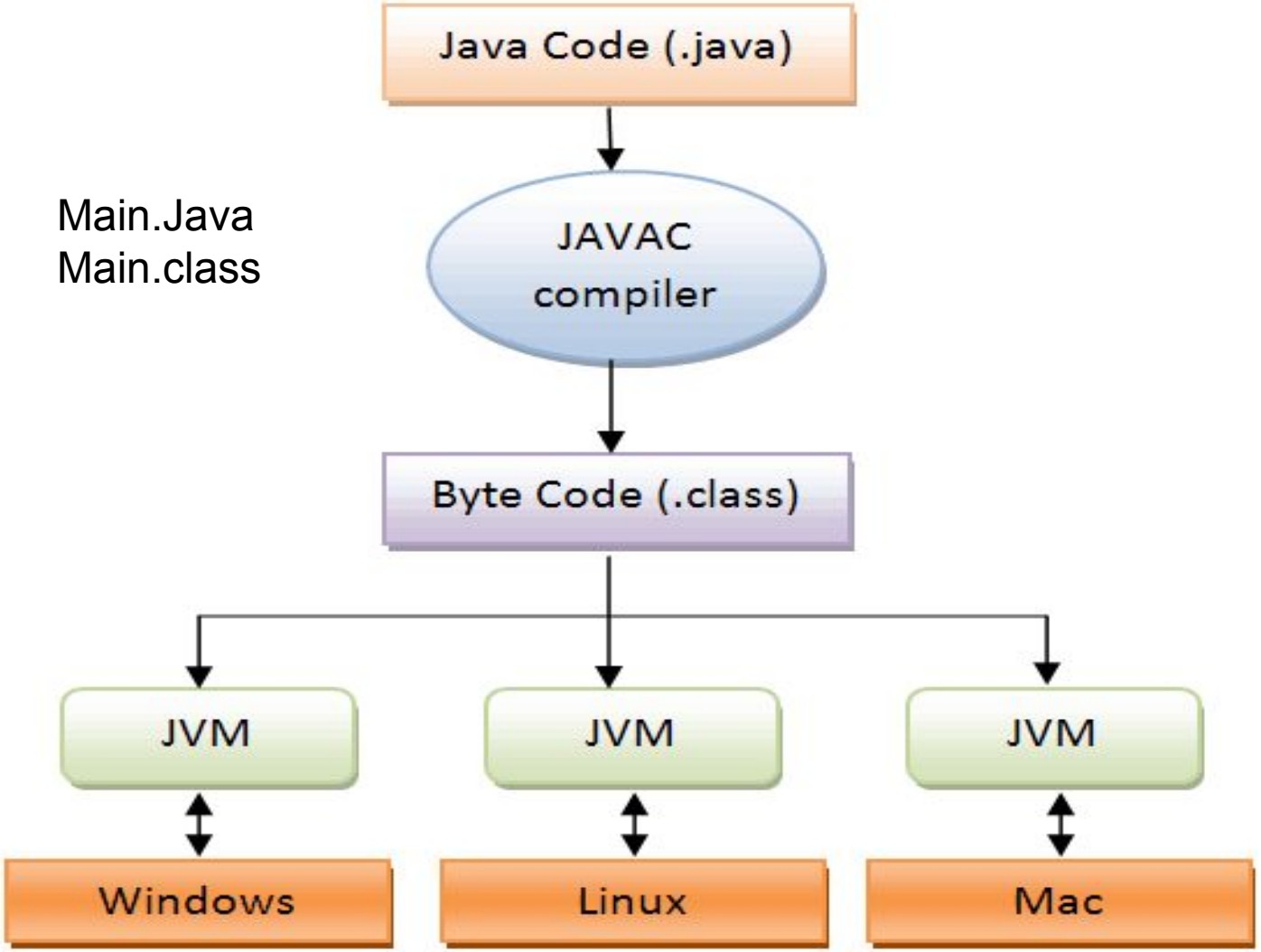
- Java is a high level programming language. A program written in high level language cannot be run on any machine directly.
- First, it needs to be translated into that particular machine language.
- It is called a virtual machine because it doesn't physically exist.
- The **javac compiler** does this thing, it takes java program (.java file containing source code) and translates it into machine code (referred as byte code or .class file).
- Java Virtual Machine (JVM) is a virtual machine that resides in the real machine (your computer) and the **machine language for JVM is byte code**.

# Java Virtual Machine (JVM)

- This makes it easier for compiler as it has to generate byte code for JVM rather than different machine code for each type of machine.
- JVM executes the byte code generated by compiler and produce output.
- **JVM is the one that makes java platform independent**.

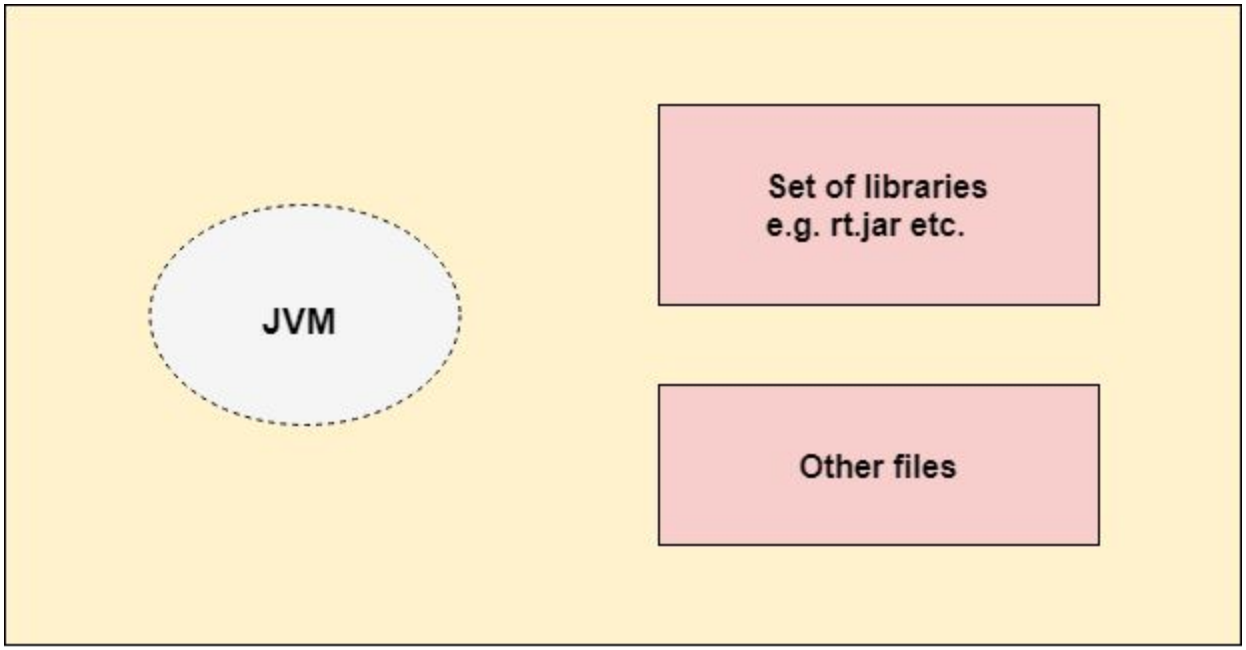
# Java Virtual Machine (JVM)

Main.Java
Main.class



# JRE

- JRE is an acronym for Java Runtime Environment.
- The Java Runtime Environment is a set of software tools which are used for developing Java applications.
- It is the implementation of JVM.
- It physically exists.
- It contains a set of libraries + other files that JVM uses at runtime.




JRE

# JDK (Java Development kit)

- The Java Development Kit (JDK) is a software development environment which is used to develop Java applications
- It contains JRE + development tools.
- The JDK contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc.

# A program Structure

**A.java**

```
public class A{
  public static void main(String[] args)
  {
    System.out.println(“Hello Word");
  }  //End of main

}
```

- **Output:** Hello world

https://ideone.com/Y0vB4d
https://ideone.com/30UNmT

# Keys

- A java file can have any number of classes but it can have only one public class and the file name should be same as public class name.

- public: This makes the main method public that means that we can call the method from outside the class.

- static: We do not need to create object for static methods to run. They can run itself.

- void: It does not return anything.

- main: It is the method name. This is the entry point method from which the JVM can run your program.

# Keys

- (String[] args): Used for command line arguments that are passed as strings.
- System.out.println(“Hello World"); : This method prints the contents inside the double quotes into the console and inserts a newline after.

# Second Program in Java

```
import java.util.Scanner;

public class Demo {
   public static void main(String[] args) {

      Scanner b= new Scanner(System.in);
      System.out.print("Enter any number: ");

      int num = b.nextInt();



      System.out.println("The number entered by user: “+num);

   }
}
```

# Object, Class in Java

- **Object** − Objects have states and behaviors.
- **Object** is a bundle of data and its behavior(often known as methods).
- Objects have two characteristics: They have **states** and **behaviors**.

- Example: A dog has states - color, name, breed as well as behavior such as wagging their tail, barking, eating.

- **Object**: House
  Variable/**State**: Address, Color, Area
  Method/**Behavior**: Open door, close door

- **Object**: Car
  **State**: Color, Brand, Weight, Model
  **Behavior**: Break, Accelerate, Slow Down, Gear change.
- So, An object is an instance of a class.

# Object, Class in Java

- **Class** : A class can be defined as a template/blueprint that describes the behavior/state that the object of its type supports.

- **Methods** : A method is basically a behavior. A class can contain many methods. It is in methods where the logics are written, data is manipulated and all the actions are executed.

- **Instance Variables** : Each object has its unique set of instance variables. An object's state is created by the values assigned to these instance variables.

```
Class Man{

string name;
Float height;
Float wieght;

Void eat();
Void run()
Void sleep()
}

Man man1;
Man man2;
```

# Object, Class in Java

```
class House
{  String address;
    String color;
    double Area;
    void openDoor()
        {

        }
    void closeDoor()
        {

        }
}
House mehed's house;
House hamze's house;
```

# Basic Syntax in Java

- There cannot be two public classes in a single Java program.
- **Case Sensitivity** − Java is case sensitive, which means identifier **World** and **world** would have different meaning in Java.
- **Class Names** − For all class names the first letter should be in Upper Case. If several words are used to form a name of the class, each inner word's first letter should be in Upper Case.
- **Example:** *class HelloWorld*
- **Method Names** − All method names should start with a Lower Case letter. If several words are used to form the name of the method, then each inner word's first letter should be in Upper Case.
- **Example:** *public void myMethodName()*

# Basic Syntax in Java

- **Program File Name :** Name of the program file should exactly match the class name.
- When saving the file, you should save it using the class name (Remember Java is case sensitive) and append '.java' to the end of the name (if the file name and the class name do not match, your program will not compile).
- **Example:** Assume 'HelloWorld' is the class name. Then the file should be saved as ' HelloWorld.*java*'
- **White Space :** Variables naming cannot contain white spaces, for example: int num ber = 100; is invalid
- **Variable Name :** Variable name can begin with special characters such as $ and _

# Basic Syntax in Java

- As per the java coding standards the variable name should begin with a lower case letter, for example

  >> int number;

- For lengthy variables names that has more than one words do it like this:
- int smallNumber;
- int bigNumber;

- (start the second word with capital letter).

# Identifier in Java

- All Java components require names. Names used for classes, variables, and methods are called **identifiers**.
- All identifiers should begin with a letter (A to Z or a to z), currency character ($) or an underscore (_).
- After the first character, identifiers can have any combination of characters.
- A keyword cannot be used as an identifier.
- Most importantly, identifiers are case sensitive.

- Examples of legal identifiers: age, $salary, _value, __1_value.
- Examples of illegal identifiers: 123abc, -salary.

# Modifier in Java

- Like other languages, it is possible to modify classes, methods, etc., by using modifiers. There are two categories of modifiers :
- **Access Modifiers** – default, public , protected, private
- **Non-access Modifiers** – final, abstract, strict

# Thank You for your patience.